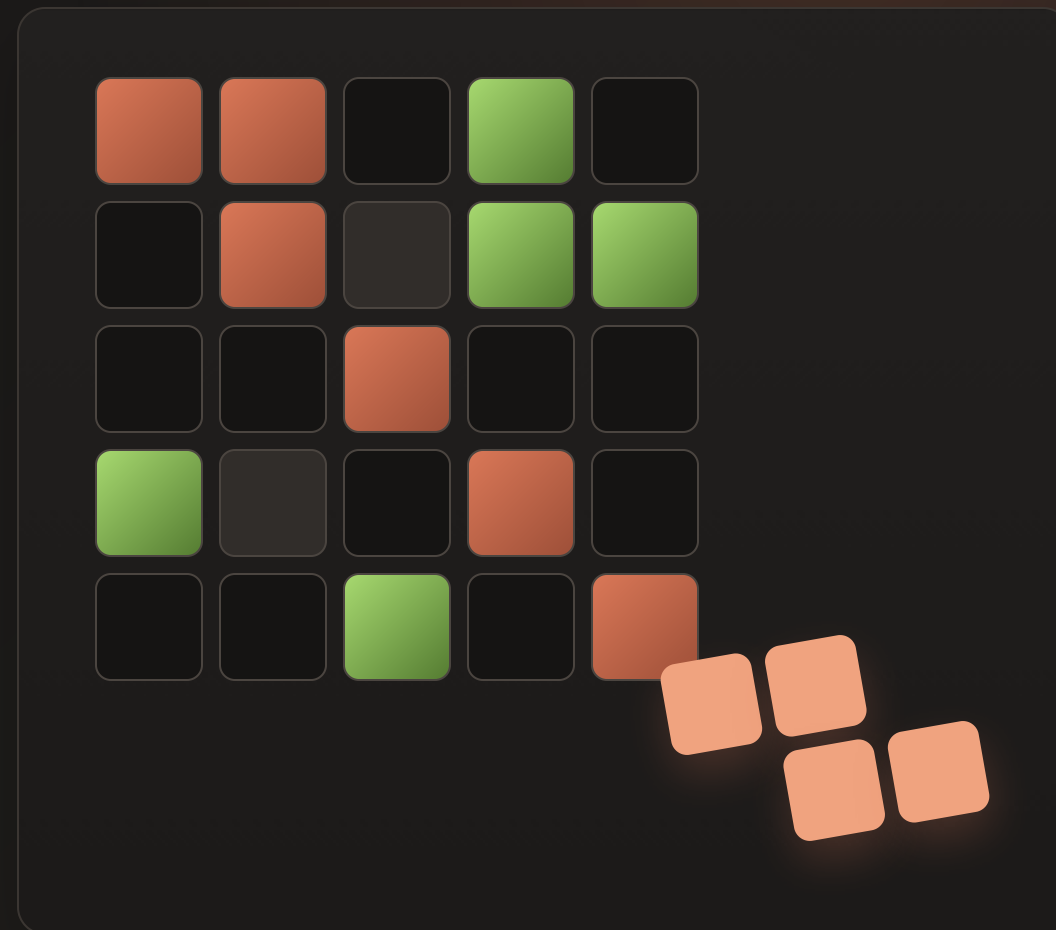


PUZZLE GAME / DESKTOP APP

Endfield Block Game

以「源石電路修復」為靈感的 Nonogram-like
拼圖小遊戲



遊戲目標

CORE RULE

把零件放對，讓提示數字成立

- 拖曳多邊形零件到棋盤
- 每列、每行都有指定顏色格數
- 所有 row / col count 剛好相等就過關
- 沒有分數，只有 pass / not yet

技術選型與責任

Backend

C++20 + CMake

關卡解析、規則判斷、勝利條件、solver。

Desktop

Electron

桌面視窗、檔案對話框、啟動 C++ process。

UI

Vue 3 + TypeScript

畫面、拖曳、提示 overlay、音效、關卡設計器。

重點：不使用遊戲引擎，因為本專案更像規則明確的互動工具。

架構：C++ 是唯一真相來源



IPC 與拖曳設計

NDJSON over stdin/stdout

```
{"id":1,"op":"place",  
  "pieceId":0,  
  "row":2,"col":3,"rot":1}
```

每個 request 用 `id` 對回 response。

拖曳不每幀問 C++

- 拖曳中：前端即時預覽
- 放開時：C++ 判斷合法性
- 成功後：重新抓後端 state

核心功能

關卡格式

純文字描述棋盤、row/col 目標、固定格、不可放置格與零件。

放置判斷

檢查越界、blocked、fixed、其他零件重疊。

Solver

回溯搜尋，自動找出一組可過關的放置方式。

關卡設計器

編輯棋盤與零件，並用 C++ solver 驗證是否可解。

Solver 與關卡設計器

Solver 做什麼

1. 保留 fixed / blocked 格
2. 產生不重複旋轉
3. 大零件優先放
4. 超過 row / col 目標就剪枝

設計器怎麼避免無解



AI 協作與開發流程

AI 加速的部分

- Vue component 與 CSS
- Electron IPC 樣板
- 跨語言資料結構整理
- 文件與報告整理

人工把關的部分

- C++ 必須負責核心規則
- 前後端責任邊界
- 實際操作測試與 demo 驗收
- 不把規則搬到 TypeScript

Demo 測試流程

1

載入測資

`backend/tests/Example*.txt`

2

操作棋盤

拖曳、旋轉、合法 / 非法放置、
重置。

3

展示功能

提示、AI幫你放、過關判定。

Load

既有關卡



Play

拖曳旋轉



Auto Solve

solver 展示



Design

自訂關卡

TAKEAWAY

規則放後端，體驗 交前端

- C++ 集中管理遊戲規則與 solver
- Electron + Vue 專注桌面 UI 與互動
- 關卡設計器讓專案不只是一個單關卡 demo
- AI 用來加速實作，但不取代架構判斷